UNIVERSIDADE FEDERAL DE UBERLÂNDIA
Faculdade de Computação – FACOM
Bacharelado em Sistemas de Informação

FACOM32503 – Programação para Dispositivos Móveis
Prof. Cláudio C. Rodrigues

Sistema de Avaliação Comparativa Multicritério de Produtos (AHP-TOPSIS)

Aplicativo *Escolha Certa* – Kotlin/Android

Fase 1 – Requisitos e Modelagem

Integrante	**Função**
Eliézer Evangelista Silva	Líder do Projeto e Designer/UX
Luiz Eduardo Rosa Sonoda	Engenheiro de PO
Kamylla Machado Rezende	Desenvolvedora Android
Paulo Victor Matias Ferreira	Engenheiro de Dados

Uberlândia – MG
Novembro de 2026

Sumário

1 Introdução **2**
1.1 Contextualização . . . 2
1.2 Problema . . . 2
1.3 Objetivos . . . 2

2 Organização da Equipe **3**

3 Escopo do Aplicativo **4**
3.1 Funcionamento geral . . . 4
3.2 Categorias e critérios . . . 4
3.3 Produtos iniciais . . . 5
3.4 Extensibilidade . . . 5
3.5 Funcionalidades do aplicativo . . . 5

4 Fundamentação Matemática **6**
4.1 Etapa 1 – AHP (pesos e consistência) . . . 6
4.2 Etapa 2 – TOPSIS (ranqueamento) . . . 7

5 Requisitos **7**
5.1 Requisitos funcionais . . . 7
5.2 Requisitos não funcionais . . . 8

6 Arquitetura do Sistema **9**
6.1 Fluxo de dados . . . 10

7 Modelagem de Dados **10**
7.1 Diagrama de classes . . . 10
7.2 Entidades em Kotlin . . . 11
7.3 Base remota – API em JSON . . . 12

8 Navegação **13**

9 Protótipos de Tela **14**
9.1 Identidade visual . . . 14
9.2 Telas . . . 15

10 Gestão do Projeto **18**

10.1 Ferramentas 18

10.2 Cronograma 18

11 Estado Atual do Projeto **19**

11.1 Artefatos entregues 19

12 Considerações e Próximos Passos **20**

Referências **20**

1 Introdução

Este documento apresenta a Fase 1 (Requisitos e Modelagem) do projeto da disciplina Programação para Dispositivos Móveis. O objetivo é desenvolver o *Escolha Certa*, um aplicativo Android modular em **Kotlin**, com arquitetura **MVVM** e interface em **Jetpack Compose**, que realiza a avaliação comparativa de produtos por meio da integração dos métodos multicritério **AHP** (*Analytic Hierarchy Process*) e **TOPSIS** (*Technique for Order of Preference by Similarity to Ideal Solution*).

1.1 Contextualização

A escolha entre produtos envolve avaliar, ao mesmo tempo, atributos frequentemente conflitantes, como preço, desempenho, autonomia de bateria e peso. A Análise de Decisão Multicritério (MCDA) transforma esses *trade-offs* em uma decisão estruturada. Entre as técnicas da literatura, AHP e TOPSIS estão entre as mais consolidadas para ranqueamento e seleção de alternativas.

1.2 Problema

O processo tradicional de comparação de produtos apresenta três limitações:

1. **Subjetividade na definição de pesos:** é difícil atribuir diretamente percentuais de importância a vários critérios sem arbitrariedade ou contradição.
2. **Exaustão do AHP puro:** o AHP original exige comparações par a par entre todas as alternativas em cada critério, o que se torna inviável quando o número de produtos cresce.
3. **Ausência de pesos nativos no TOPSIS:** o TOPSIS exige pesos informados externamente e não valida a coerência do julgamento humano.

Pergunta de pesquisa: *como estruturar um modelo híbrido de apoio à decisão (AHP--TOPSIS) para comparar produtos que reduza a subjetividade na atribuição dos pesos dos critérios e mantenha a eficiência do ranqueamento diante de muitos produtos e atributos?*

1.3 Objetivos

Objetivo geral: desenvolver um aplicativo Android que implemente o modelo híbrido AHP-TOPSIS para apoio à tomada de decisão na escolha de produtos.

Objetivos específicos:

- **Modelagem matemática:** estruturar a matriz de comparação par a par de Saaty e validar a coerência lógica pela Razão de Consistência ($RC \leq 0{,}10$).
- **Algoritmo de ranqueamento:** implementar a normalização vetorial e o cálculo das distâncias euclidianas às soluções ideais positiva (A^+) e negativa (A^-).

- **Engenharia de software:** aplicar MVVM, separação de responsabilidades, interface reativa e integração com base de dados remota.
- **Gestão de projeto:** organizar o trabalho em equipe com papéis definidos, controle de versão e quadro de tarefas.

2 Organização da Equipe

O grupo é formado por quatro integrantes, cada um responsável por um papel. Os papéis cobrem as funções sugeridas no enunciado.

Integrante	**Papel**	**Responsabilidades principais**
Eliézer Evangelista Silva	Líder + Designer/UX	Coordenação geral, cronograma, quadro de tarefas, revisão de *pull requests*, integração das entregas, comunicação com o professor, identidade visual e protótipos no Figma.
Luiz Eduardo Rosa Sonoda	Engenheiro de PO	Modelagem matemática AHP/TOPSIS, implementação dos motores de cálculo (`AhpEngine` e `TopsisEngine`) e testes matriciais.
Kamylla Machado Rezende	Desenvolvedora Android	Telas em Jetpack Compose, navegação (NavHost), ViewModels e gerenciamento de estado com StateFlow.
Paulo Victor Matias Ferreira	Engenheiro de Dados	Modelagem das entidades, configuração da base remota (API em JSON consumida com Retrofit), banco local do histórico (Room), repositórios e cadastro dos produtos.

Tabela 1: Papéis e responsabilidades da equipe.

3 Escopo do Aplicativo

3.1 Funcionamento geral

O usuário escolhe uma categoria, seleciona os produtos que deseja comparar e informa, por comparações par a par entre critérios, o que é mais importante para ele. O aplicativo calcula os pesos com o AHP, valida a consistência dos julgamentos e, com esses pesos, ranqueia os produtos com o TOPSIS, destacando a melhor opção.

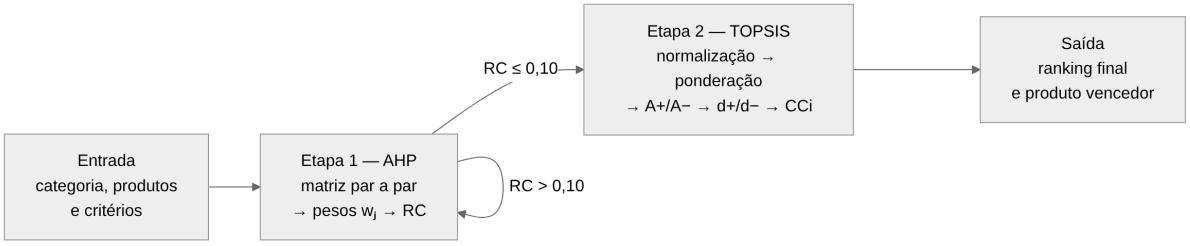


Figura 1: Fluxo metodológico do modelo híbrido AHP-TOPSIS.

3.2 Categorias e critérios

Foram selecionadas duas categorias do banco de categorias proposto no enunciado. Cada uma possui cinco critérios, classificados como **benefício** (quanto maior, melhor) ou **custo** (quanto menor, melhor).

ID	Critério	Tipo	Unidade
Categoria 1 – Notebooks Corporativos (TI/Hardware)			
C1	Desempenho (CPU/RAM)	Benefício	Pontos (benchmark)
C2	Preço de aquisição	Custo	R$
C3	Autonomia de bateria	Benefício	Horas
C4	Peso / portabilidade	Custo	kg
C5	Qualidade de construção	Benefício	Nota 1–10
Categoria 2 – Smartphones Premium (Tecnologia/Consumo)			
C1	Desempenho (Geekbench)	Benefício	Pontos
C2	Preço de venda	Custo	R$
C3	Qualidade de câmeras	Benefício	Nota 1–10
C4	Autonomia de bateria	Benefício	mAh
C5	Armazenamento interno	Benefício	GB

Tabela 2: Critérios de avaliação por categoria.

3.3 Produtos iniciais

Cada categoria começa com quatro produtos, um para cada perfil sugerido no enunciado. Os valores de cada critério serão levantados em fontes públicas (fabricantes, lojas e sites de *benchmark*) e cadastrados na base remota na Fase 2.

Perfil	Notebooks	Smartphones
A	Workstation – Apple MacBook Pro 14”	*Flagship* – Samsung Galaxy S26 Ultra
B	Ultrabook – Lenovo ThinkPad X1 Carbon	Foco em câmera – Apple iPhone 17 Pro
C	Intermediário – Dell Inspiron 14	Custo-benefício – Samsung Galaxy A56
D	Robusto – Lenovo ThinkPad T14	Bateria – Motorola Moto G (linha Power)

Tabela 3: Produtos cadastrados inicialmente.

3.4 Extensibilidade

Uma decisão central do projeto é que o aplicativo **não tem quantidade fixa de produtos**. Os produtos são lidos dinamicamente da base remota, de modo que:

- novos produtos podem ser cadastrados apenas editando o arquivo JSON da base remota, sem alterar o código nem publicar nova versão do aplicativo;
- novas categorias podem ser criadas com a mesma estrutura (nome, ícone, cinco critérios e lista de produtos);
- o usuário escolhe quais produtos de uma categoria deseja comparar (no mínimo dois), mantendo o ranking legível mesmo com muitos produtos cadastrados.

O TOPSIS opera com qualquer número m de alternativas. O número de critérios n é fixo em cinco por categoria, dentro do limite da tabela de Índice Aleatório de Saaty ($n \leq 10$).

3.5 Funcionalidades do aplicativo

Além do fluxo principal de avaliação, o aplicativo foi projetado com funcionalidades que tornam a ferramenta mais completa e didática:

- **Busca e filtros de produtos** por nome, fabricante, modelo e perfil, essenciais quando a categoria tem muitos produtos cadastrados;
- **Seleção dos produtos a comparar**, com tela de detalhes de cada produto;
- **Validação visual da consistência**, com alerta e bloqueio quando $RC > 0{,}10$;
- **Matriz de decisão** exibida no resumo antes do processamento;

- **Análise de sensibilidade ("E se?")**: o usuário altera os pesos e observa, em tempo real, como o ranking muda, identificando se a escolha é robusta;
- **Histórico de avaliações** salvo automaticamente no aparelho;
- **Exportação e compartilhamento** do resultado em PDF;
- **Ajuda "Como funciona"**, explicando o AHP e o TOPSIS de forma visual;
- **Ajustes**: tema, atualização do catálogo e limpeza do histórico;
- **Navegação inferior** com as abas Início, Histórico e Ajustes.

4 Fundamentação Matemática

4.1 Etapa 1 – AHP (pesos e consistência)

1. Monta-se a matriz de comparação par a par $A = [a_{ij}]_{n\times n}$ com a Escala Fundamental de Saaty, em que a_{ij} indica a importância do critério i sobre o critério j, com $a_{ii} = 1$ e $a_{ji} = 1/a_{ij}$.

Valor	Significado
1	Igual importância
3	Importância moderada
5	Importância forte
7	Importância muito forte
9	Importância extrema
2, 4, 6, 8	Valores intermediários

Tabela 4: Escala Fundamental de Saaty.

2. Calcula-se o vetor de pesos w normalizando as colunas e tirando a média das linhas:

$$\bar{a}_{ij} = \frac{a_{ij}}{\sum_{k=1}^{n} a_{kj}}, \qquad w_i = \frac{1}{n}\sum_{j=1}^{n} \bar{a}_{ij}, \qquad \sum_{i=1}^{n} w_i = 1$$

3. Determina-se o autovalor máximo: $\lambda_{max} = \frac{1}{n}\sum_{i=1}^{n} \frac{(A\cdot w)_i}{w_i}$

4. Calculam-se o Índice de Consistência e a Razão de Consistência:

$$IC = \frac{\lambda_{max} - n}{n-1}, \qquad RC = \frac{IC}{IR}$$

em que IR é o Índice Aleatório de Saaty. Para $n = 5$, $IR = 1{,}12$.

5. **Regra de decisão:** se $RC \leq 0{,}10$, os julgamentos são válidos. Caso contrário, o aplicativo bloqueia o avanço e solicita a revisão da matriz.

4.2 Etapa 2 – TOPSIS (ranqueamento)

1. **Normalização vetorial:** $r_{ij} = x_{ij} \,/\, \sqrt{\sum_{k=1}^{m} x_{kj}^2}$
2. **Matriz ponderada:** $v_{ij} = w_j \cdot r_{ij}$, com w_j validado no AHP.
3. **Soluções ideais:** para critérios de benefício, $A_j^+ = \max_i v_{ij}$ e $A_j^- = \min_i v_{ij}$; para critérios de custo, $A_j^+ = \min_i v_{ij}$ e $A_j^- = \max_i v_{ij}$.
4. **Distâncias euclidianas:**

$$d_i^+ = \sqrt{\sum_{j=1}^{n} (v_{ij} - A_j^+)^2}, \qquad d_i^- = \sqrt{\sum_{j=1}^{n} (v_{ij} - A_j^-)^2}$$

5. **Coeficiente de proximidade relativa:** $CC_i = d_i^- / (d_i^+ + d_i^-)$, com $0 \leq CC_i \leq 1$. As alternativas são ordenadas de forma decrescente por CC_i; a de maior valor é a recomendada.

5 Requisitos

5.1 Requisitos funcionais

ID	Descrição	Prioridade
RF01	Exibir tela de abertura (*splash screen*) animada com logotipo e nome do aplicativo.	Média
RF02	Listar as categorias de produtos em *cards* com ícone e nome, carregadas da base remota.	Alta
RF03	Listar os produtos de uma categoria em *cards* com foto, nome, fabricante, especificações e perfil.	Alta
RF04	Buscar produtos por nome, fabricante ou modelo e filtrá-los por perfil.	Média
RF05	Exibir os detalhes de um produto: foto, nome, modelo, fabricante, especificações e valores dos cinco critérios.	Alta
RF06	Permitir ao usuário selecionar quais produtos da categoria serão comparados (mínimo de dois).	Alta
RF07	Permitir a entrada da matriz de comparação par a par dos cinco critérios na escala de Saaty (1–9), preenchendo automaticamente os valores recíprocos.	Alta
RF08	Calcular automaticamente o vetor de pesos, o autovalor máximo (λ_{max}), o Índice de Consistência (IC) e a Razão de Consistência (RC).	Alta
RF09	Exibir alerta visual e bloquear o avanço quando a matriz for inconsistente ($RC > 0{,}10$).	Alta

ID	Descrição	Prioridade
RF10	Exibir, por critério, os valores de cada produto selecionado, permitindo ajustá-los (botões $-/+$), com indicador de progresso.	Média
RF11	Exibir tela de resumo com produtos selecionados, pesos, RC e a matriz de decisão, com botão "Processar".	Alta
RF12	Calcular a matriz normalizada e a matriz ponderada do TOPSIS.	Alta
RF13	Exibir o ranking final com d^+, d^- e CC_i de cada produto, destacando graficamente o vencedor.	Alta
RF14	Exibir análise detalhada com gráfico de CC_i, matriz normalizada ponderada e pesos.	Média
RF15	Permitir a análise de sensibilidade: alterar os pesos e recalcular o ranking em tempo real, sinalizando mudanças de posição.	Média
RF16	Salvar automaticamente cada avaliação concluída e listá-las na tela de Histórico, com filtro por categoria, reabertura e exclusão.	Média
RF17	Exportar e compartilhar o resultado da avaliação em PDF.	Baixa
RF18	Exibir tela de ajuda "Como funciona" explicando o AHP e o TOPSIS.	Baixa
RF19	Exibir tela de Ajustes com tema, atualização do catálogo e limpeza do histórico.	Baixa
RF20	Oferecer navegação inferior entre Início, Histórico e Ajustes.	Média
RF21	Permitir iniciar uma nova avaliação a partir da tela de resultados.	Média
RF22	Exibir mensagens de carregamento e de erro quando não houver conexão com a base remota.	Média

Tabela 5: Requisitos funcionais.

5.2 Requisitos não funcionais

ID	Descrição
RNF01	**Plataforma:** aplicativo Android nativo em Kotlin, com interface em Jetpack Compose e Material Design 3.
RNF02	**Arquitetura:** padrão MVVM com separação em camadas de UI, apresentação, domínio e dados.
RNF03	**Usabilidade:** interface intuitiva e responsiva, com fluxo de no máximo um toque entre etapas consecutivas.
RNF04	**Desempenho:** cálculos matriciais executados instantaneamente (menos de 1 segundo) e fora da *thread* principal.

ID	Descrição
RNF05	**Dados remotos:** produtos e categorias obtidos de uma API remota em formato JSON, consumida com Retrofit, sem dados fixos no código.
RNF06	**Extensibilidade:** inclusão de novos produtos e categorias sem alteração de código.
RNF07	**Manutenibilidade:** código modular, documentado e com testes unitários dos motores de cálculo.
RNF08	**Versionamento:** controle de versão com Git em repositório compartilhado no GitHub, com *branches* por funcionalidade e *pull requests* revisados.
RNF09	**Gestão:** tarefas organizadas no GitHub Projects.
RNF10	**Persistência local:** histórico armazenado com Room (SQLite) e preferências com DataStore, disponíveis mesmo sem conexão.
RNF11	**Acessibilidade:** contraste adequado entre texto e fundo, áreas de toque de no mínimo 48 dp e textos descritivos nos ícones.

Tabela 6: Requisitos não funcionais.

6 Arquitetura do Sistema

O aplicativo segue a arquitetura recomendada para aplicações Android reativas, organizada em quatro camadas (Figura 2).

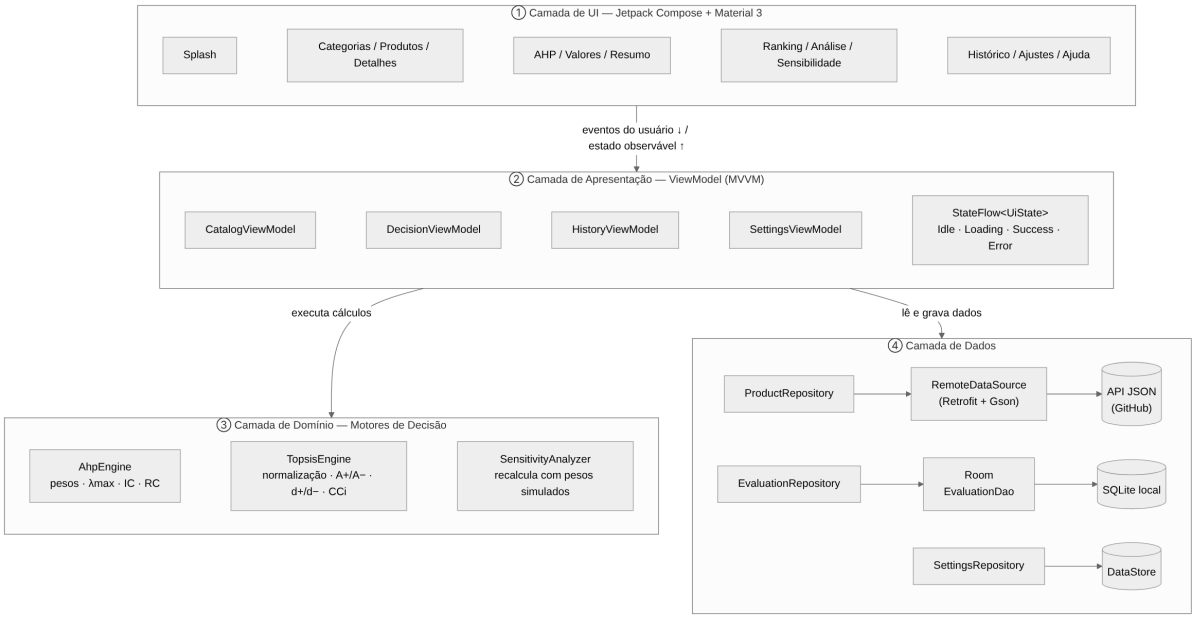


Figura 2: Arquitetura MVVM em camadas do *Escolha Certa*.

1. **Camada de UI (Jetpack Compose + Material 3):** renderiza as telas, captura as ações do usuário e reage às mudanças de estado. Não contém regra de negócio.

2. **Camada de Apresentação (ViewModel):** mantém e expõe o estado da tela por meio de `StateFlow`, recebe os eventos da UI e orquestra as chamadas aos motores e ao repositório usando *coroutines*, sem travar a *main thread*. Os estados possíveis são *Idle*, *Loading*, *Success* e *Error*.
3. **Camada de Domínio (motores de decisão):** contém o `AhpEngine` (pesos, λ_{max}, IC e RC), o `TopsisEngine` (normalização, soluções ideais, distâncias e CC_i) e o `SensitivityAnalyzer`, que reexecuta o TOPSIS com pesos simulados. São classes Kotlin puras, independentes do Android, o que facilita os testes unitários.
4. **Camada de Dados:** o `ProductRepository` consulta a API remota por meio do `RemoteDataSource` (Retrofit) e converte o JSON em modelos Kotlin; o `EvaluationRepository` grava e lê o histórico no banco local (Room); o `SettingsRepository` guarda as preferências com DataStore.

6.1 Fluxo de dados

1. A tela solicita as categorias; o ViewModel chama o repositório, que consulta a API remota e devolve a lista. O estado passa de *Loading* para *Success*, e a UI se redesenha.
2. Na tela do AHP, cada alteração de *slider* é enviada ao `DecisionViewModel`, que recalcula pesos e RC com o `AhpEngine` e atualiza o estado em tempo real.
3. Ao tocar em "Processar", o ViewModel envia a matriz de decisão e os pesos ao `TopsisEngine` e publica a lista ordenada de `DecisionResult`; em seguida, a avaliação é salva no histórico pelo `EvaluationRepository`.
4. Na análise de sensibilidade, cada alteração de peso aciona o `SensitivityAnalyzer`, e o ranking simulado é atualizado sem alterar a avaliação original.

7 Modelagem de Dados

7.1 Diagrama de classes

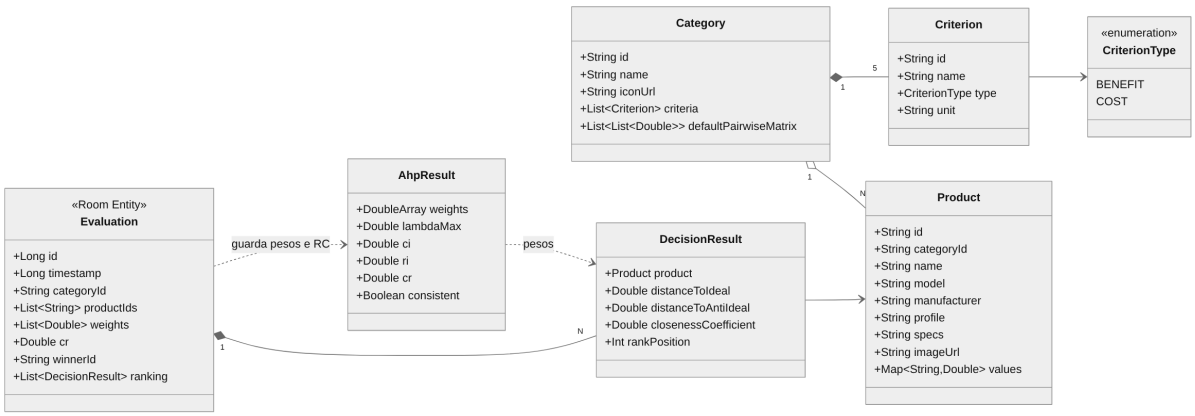


Figura 3: Diagrama de classes das entidades do aplicativo.

7.2 Entidades em Kotlin

```kotlin
enum class CriterionType { BENEFIT, COST }

data class Criterion(
    val id: String, // "c1" ... "c5"
    val name: String, // ex.: "Preço de aquisição"
    val type: CriterionType, // BENEFIT ou COST
    val unit: String // ex.: "R$", "kg", "h"
)

data class Category(
    val id: String, // ex.: "notebooks"
    val name: String,
    val iconUrl: String,
    val criteria: List<Criterion>, // sempre 5
    val defaultPairwiseMatrix: List<List<Double>> // sugestão inicial
)

data class Product(
    val id: String,
    val categoryId: String,
    val name: String,
    val model: String,
    val manufacturer: String,
    val profile: String, // ex.: "Workstation"
    val specs: String, // ex.: "Chip M5, 24 GB"
    val imageUrl: String,
    val values: Map<String, Double> // "c1" -> 9800.0, "c2" -> 5300.0 ...
)

data class AhpResult(
    val weights: DoubleArray, val lambdaMax: Double,
    val ci: Double, val ri: Double, val cr: Double,
    val consistent: Boolean // cr <= 0.10
)

data class DecisionResult(
    val product: Product,
    val distanceToIdeal: Double, // d+
    val distanceToAntiIdeal: Double, // d-
    val closenessCoefficient: Double, // CCi
    val rankPosition: Int
)

@Entity(tableName = "evaluations") // histórico (Room)
data class Evaluation(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val timestamp: Long,
    val categoryId: String,
    val productIds: List<String>,
    val weights: List<Double>,
    val cr: Double,
    val winnerId: String,
    val ranking: List<DecisionResult> // salvo via TypeConverter (JSON)
)
```

7.3 Base remota – API em JSON

A base remota é uma **API em formato JSON**, seguindo o modelo apresentado no enunciado. O arquivo `dados-categorias-ahp-topsis.json` fica hospedado no repositório do projeto no GitHub e é acessado pelo aplicativo por uma URL pública, utilizando a biblioteca **Retrofit** para a requisição HTTP e o conversor **Gson** para transformar o JSON em objetos Kotlin. Como o aplicativo apenas lê os dados, essa solução atende ao escopo com baixa complexidade de configuração.

```
{
  "categories": [
    {
      "id": "notebooks",
      "name": "Notebooks Corporativos",
      "iconUrl": "https://...",
      "criteria": [
        { "id": "c1", "name": "Desempenho (CPU/RAM)", "type": "BENEFIT", "unit": "
            pontos" },
        { "id": "c2", "name": "Preço de aquisição", "type": "COST", "unit": "R$" },
        { "id": "c3", "name": "Autonomia de bateria", "type": "BENEFIT", "unit": "h" },
        { "id": "c4", "name": "Peso", "type": "COST", "unit": "kg" },
        { "id": "c5", "name": "Qualidade de construção", "type": "BENEFIT", "unit": "
            nota" }
      ],
      "defaultPairwiseMatrix": [ [1, 3, 5, 5, 3], ... ],
      "products": [
        {
          "id": "macbook-pro-14",
          "name": "MacBook Pro 14",
          "model": "M5",
          "manufacturer": "Apple",
          "imageUrl": "https://...",
          "values": { "c1": 0.0, "c2": 0.0, "c3": 0.0, "c4": 0.0, "c5": 0.0 }
        }
      ]
    },
    { "id": "smartphones", "name": "Smartphones Premium", ... }
  ]
}
```

```
interface CatalogApi {
    @GET("dados-categorias-ahp-topsis.json")
    suspend fun getCatalog(): CatalogResponse
}

data class CatalogResponse(val categories: List<Category>)
```

A lista `products` não tem tamanho fixo. Para incluir um novo produto, basta acrescentar um objeto a essa lista e salvar o arquivo no repositório; o aplicativo o exibirá automaticamente na próxima leitura, sem alteração de código (RNF06).

8 Navegação

A navegação é implementada com o **Navigation Compose** (`NavHost`), com uma rota por tela e uma barra de navegação inferior com três destinos principais: Início, Histórico e Ajustes. O `DecisionViewModel` é compartilhado entre as telas do fluxo de avaliação para manter os produtos selecionados, a matriz e os resultados.

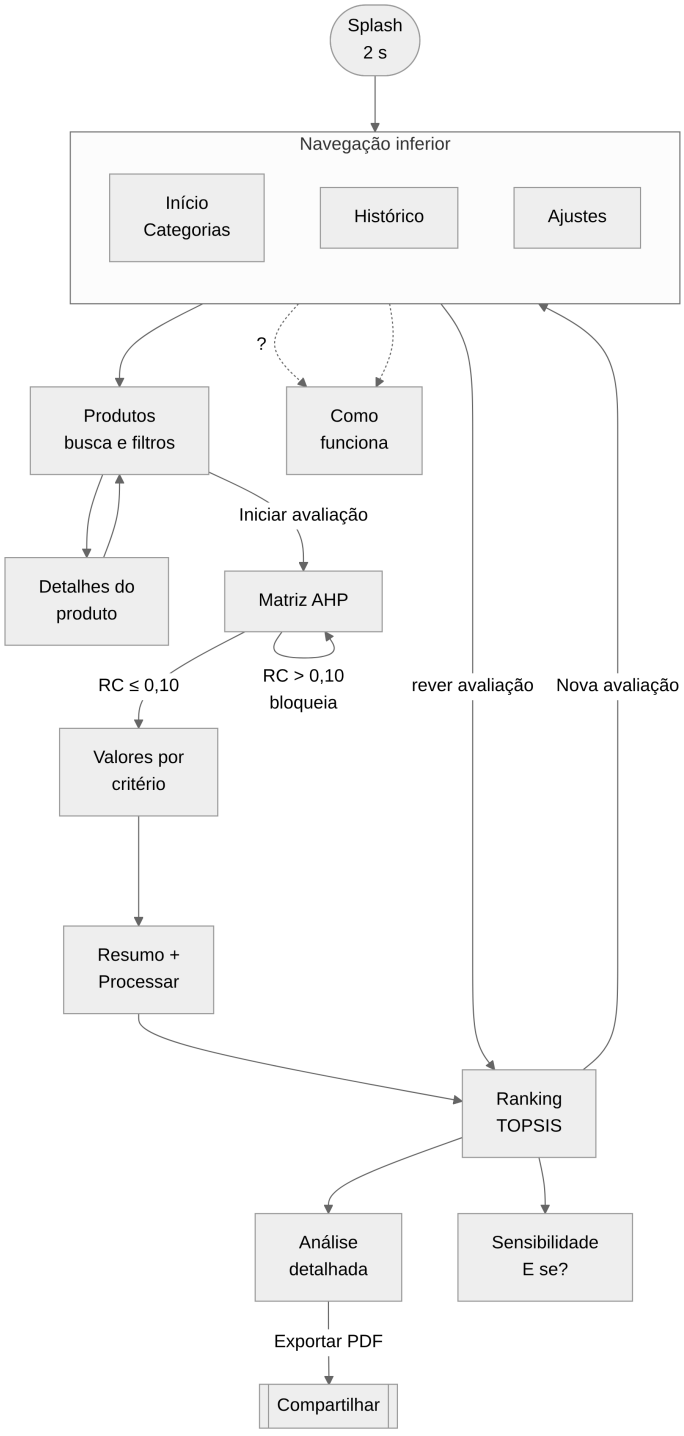


Figura 4: Diagrama de navegação entre telas.

Rota	Tela	Ação principal
`splash`	Abertura	Avança automaticamente após 2 s
`categories`	Início (categorias)	Escolher categoria; ver última avaliação
`products/{categoryId}`	Produtos	Ver detalhes ou selecionar produtos e iniciar avaliação
`product/{productId}`	Detalhes	Visualizar dados do produto
`ahp`	Matriz AHP	Comparar critérios; avança apenas com $RC \leq 0{,}10$
`values`	Valores por critério	Conferir e ajustar valores dos produtos
`summary`	Resumo	Revisar e tocar em "Processar"
`ranking`	Ranking TOPSIS	Ver vencedor e posições; sensibilidade; nova avaliação
`charts`	Análise detalhada	Ver gráficos e matriz ponderada; exportar PDF
`sensitivity`	Sensibilidade	Simular pesos e ver o ranking mudar
`history`	Histórico	Rever, filtrar e excluir avaliações salvas
`settings`	Ajustes	Tema, catálogo e limpeza do histórico
`help`	Como funciona	Explicação do AHP e do TOPSIS

Tabela 7: Rotas de navegação.

9 Protótipos de Tela

Os protótipos foram elaborados no **Figma**, no formato *Android Compact* (412×917), com navegação interativa entre as telas.

9.1 Identidade visual

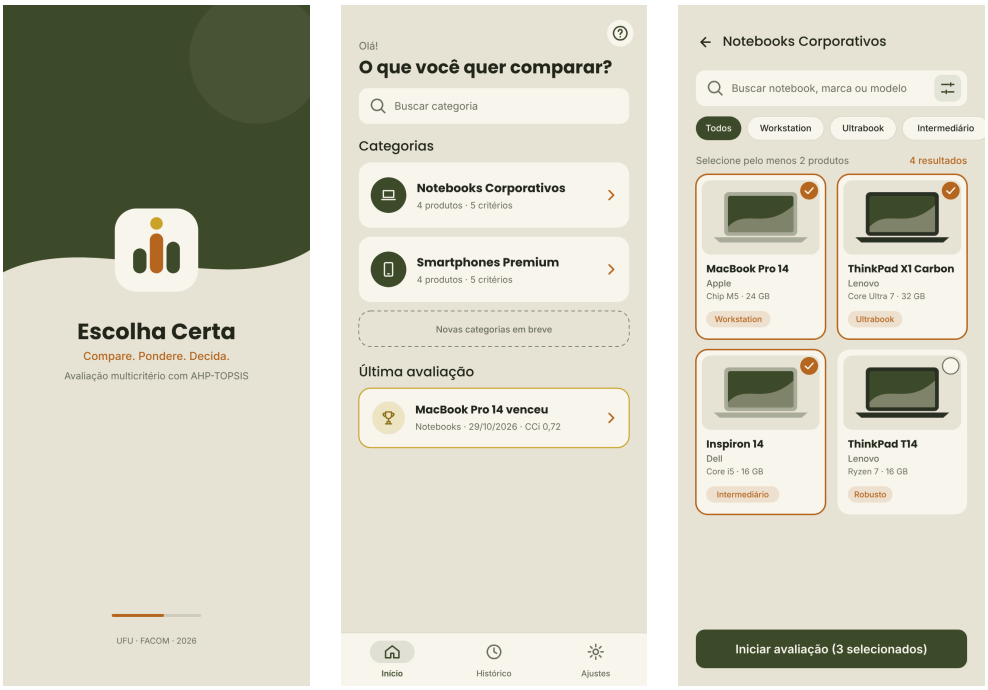
- **Paleta de cores:** paleta "Oliva & Cobre", sofisticada e fora do padrão azul comum em aplicativos. O verde-oliva escuro (primária) transmite equilíbrio e ponderação, alinhado à proposta de uma decisão racional; o cobre (secundária) destaca números e ações; o fundo creme-oliva reduz o cansaço visual em relação ao branco puro; o dourado é reservado ao produto vencedor, criando destaque imediato; verde e vermelho indicam consistência válida e inválida do AHP.
- **Tipografia:** fontes sem serifa, mais legíveis em telas pequenas – *Poppins* para títulos e *Inter* para textos e números.
- **Iconografia:** ícones do Material Symbols, padrão do Android.
- **Hierarquia:** o produto vencedor recebe *card* maior, elevação e ícone de troféu, direcionando a atenção do usuário para a escolha ótima.

Uso	Cor	Hex
Primária	Verde-oliva	#3D4A2A
Secundária	Cobre	#B5651D
Vencedor	Dourado	#C9A227
Consistente	Verde-folha	#4F7A3A
Inconsistente	Vermelho-vinho	#9E2A2B
Fundo	Creme-oliva	#E7E3D4
Superfície	Marfim	#F7F5EC
Texto	Grafite-oliva	#22261A

Tabela 8: Paleta de cores do aplicativo.

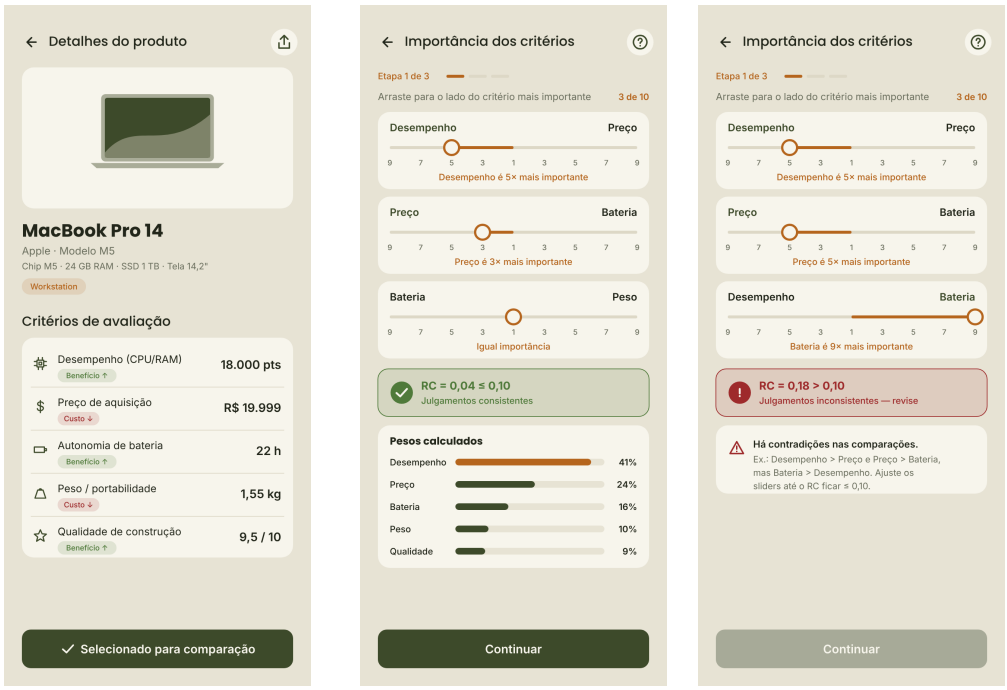
9.2 Telas

O protótipo é composto por 13 telas e um estado alternativo (5b), que mostra o bloqueio por inconsistência. Os valores exibidos são ilustrativos.



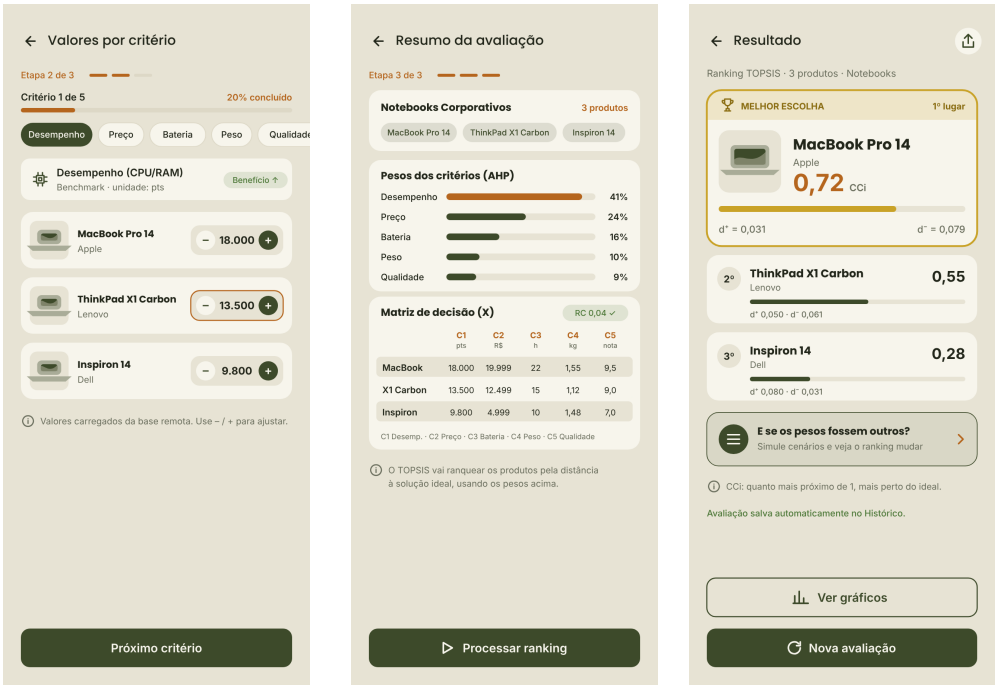
1. Abertura **2. Início (categorias)** **3. Produtos**

Figura 5: Abertura, início e lista de produtos com busca e filtros.



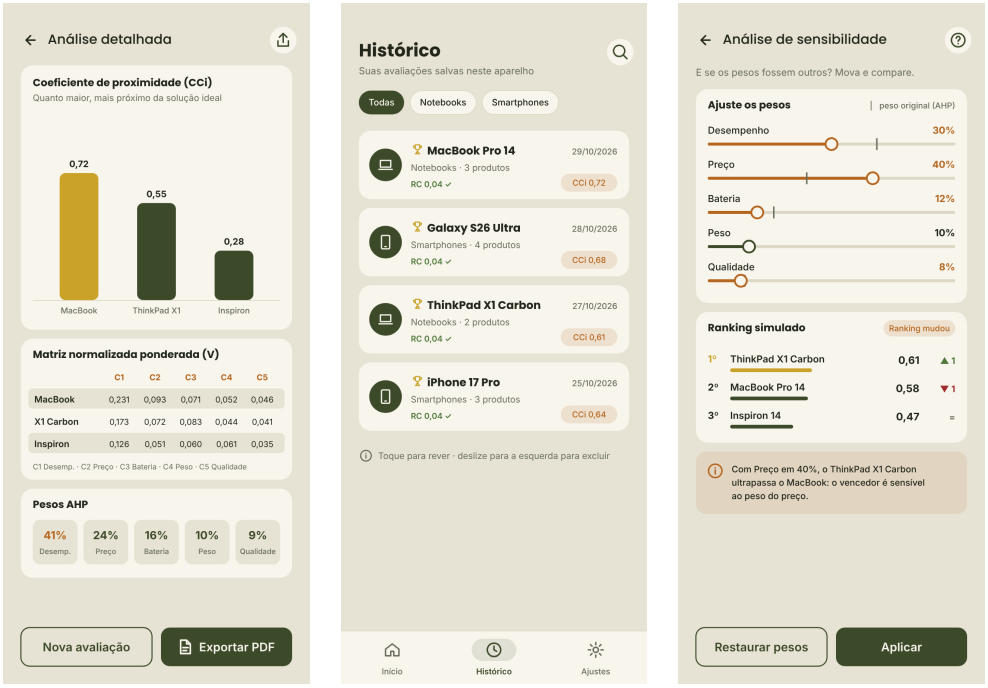
4. Detalhes do produto **5. Matriz AHP** **5b. RC inconsistente**

Figura 6: Detalhes do produto e comparação par a par (estados consistente e inconsistente).



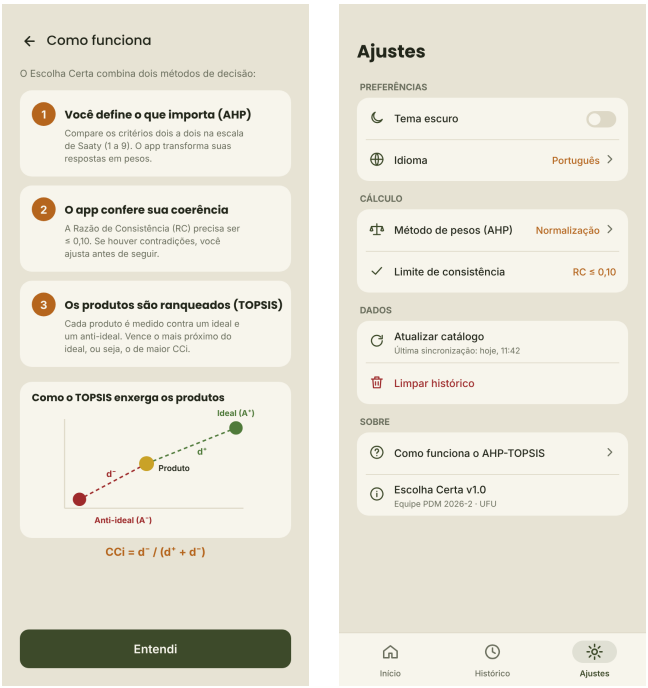
6. Valores por critério **7. Resumo** **8. Ranking TOPSIS**

Figura 7: Valores por critério, resumo com matriz de decisão e ranking.



9. Análise detalhada **10. Histórico** **11. Sensibilidade**

Figura 8: Análise detalhada com exportação, histórico e análise de sensibilidade.



12. Como funciona **13. Ajustes**

Figura 9: Ajuda e ajustes.

Link do protótipo navegável:
`https://www.figma.com/proto/TieRZW5OSy6Juz26yCvBd5/AHP-TOPSIS-%E2%80%93-P`

rot%C3%B3tipos?node-id=18-1963&p=f&t=ecky9ucgF4jY4nY8-0&scaling=scale-down&content-scaling=fixed&page-id=1%3A2

10 Gestão do Projeto

10.1 Ferramentas

- **Repositório:** `https://github.com/eliezersilva1994-eng/ahp-topsis-app`
- **Quadro de tarefas:** GitHub Projects (*AHP-TOPSIS – Cronograma*), com colunas *Todo*, *In Progress* e *Done* e tarefas separadas por fase.
- **Fluxo de trabalho:** *branch* por funcionalidade, *pull request* revisado pelo líder e README atualizado com instruções de compilação.
- **Comunicação:** reuniões semanais curtas (15–20 min) para revisar o progresso.

10.2 Cronograma

Fase	Período	Atividades	Entrega
1. Requisitos e modelagem	até 10/11	Requisitos, arquitetura MVVM, modelo de dados, navegação e protótipos no Figma	Documento e vídeo
2. Desenvolvimento e integração	11/11 – 27/11	Motores AHP/TOPSIS e sensibilidade, telas em Compose, navegação, API remota, histórico (Room) e exportação PDF	Versão funcional
3. Testes e consolidação	28/11 – 07/12	Testes unitários e de usabilidade, ajustes de layout e desempenho	App final
4. Apresentação final	até 11/12	Demonstração do aplicativo e dos resultados	Apresentação

Tabela 9: Cronograma do projeto.

11 Estado Atual do Projeto

Ao final da Fase 1, a equipe concluiu o planejamento e a modelagem do aplicativo *Escolha Certa*. As atividades realizadas foram:

- definição do escopo: duas categorias (Notebooks Corporativos e Smartphones Premium), cinco critérios por categoria e estrutura extensível de produtos;
- levantamento de 22 requisitos funcionais e 11 requisitos não funcionais;
- modelagem da arquitetura MVVM em quatro camadas, do modelo de dados e da base remota (API em JSON) e local (Room);
- especificação da navegação (NavHost e barra inferior) e diagrama de navegação;
- definição da identidade visual e prototipação de 13 telas navegáveis no Figma;
- criação do repositório no GitHub e do quadro de tarefas das duas fases no GitHub Projects, com a divisão de responsabilidades da Fase 2.

Nenhuma linha de código do aplicativo foi escrita nesta fase, conforme o cronograma: a implementação começa na Fase 2.

11.1 Artefatos entregues

#	**Artefato**	**Formato e acesso**
1	Relatório da Fase 1 (este documento)	PDF, anexado no MS Teams
2	Protótipo navegável (13 telas + estado 5b)	Figma: `https://www.figma.com/proto/TieRZW5OSy6Juz26yCvBd5/AHP-TOPSIS-%E2%80%93-Prot%C3%B3tipos?node-id=18-1963&p=f&t=ecky9ucgF4jY4nY8-0&scaling=scale-down&content-scaling=fixed&page-id=1%3A2`
3	Diagramas de fluxo, arquitetura, classes e navegação	Imagens incluídas nas Seções 3, 6, 7 e 8
4	Telas do protótipo exportadas	Imagens incluídas na Seção 9
5	Apresentação da Fase 1	Slides em PDF, anexados no MS Teams
6	Vídeo explicativo do estado do projeto	YouTube: *[colar aqui o link do vídeo no YouTube]*
7	Repositório e quadro de tarefas	GitHub: `https://github.com/eliezersilva1994-eng/ahp-topsis-app`

Tabela 10: Artefatos digitais produzidos na Fase 1.

12 Considerações e Próximos Passos

Esta fase definiu o escopo, os requisitos, a arquitetura, o modelo de dados, a navegação e os protótipos do *Escolha Certa*. Na Fase 2, a equipe implementará os motores `AhpEngine`, `TopsisEngine` e `SensitivityAnalyzer` com testes unitários, as telas em Jetpack Compose a partir dos protótipos, a navegação com NavHost, a integração com a API remota e o histórico local com Room, concluindo com testes de usabilidade e a apresentação do aplicativo funcionando.

Melhorias futuras (fora do escopo obrigatório):

- **Contas de usuário:** login para sincronizar o histórico entre aparelhos;
- **Comparação de cenários salvos:** comparar lado a lado duas simulações de sensibilidade;
- **Novas categorias:** inclusão das categorias Veículos Elétricos e Provedores de Nuvem, aproveitando a mesma estrutura de dados.

Referências

SAATY, T. L. *The Analytic Hierarchy Process*. New York: McGraw-Hill, 1980.

HWANG, C. L.; YOON, K. *Multiple Attribute Decision Making: Methods and Applications*. Berlin: Springer-Verlag, 1981.

ANDROID DEVELOPERS. *Guide to app architecture*. Disponível em: `https://developer.android.com/topic/architecture`.

ANDROID DEVELOPERS. *Jetpack Compose*. Disponível em: `https://developer.android.com/compose`.

ANDROID DEVELOPERS. *Save data in a local database using Room*. Disponível em: `https://developer.android.com/training/data-storage/room`.

SQUARE. *Retrofit – A type-safe HTTP client for Android and Java*. Disponível em: `https://square.github.io/retrofit/`.

RODRIGUES, C. C. *Sistema de Avaliação Comparativa Multicritério de Produtos (AHP-TOPSIS)*. Enunciado do projeto, FACOM/UFU, 2026.